

Binary Tree using linked list

```
#include <iostream>
using namespace std;

struct node {
    int item;
    node* left;
    node* right;
};

void inorderTraversal(node* root) {
    if (!root) return;
    inorderTraversal(root->left);
    cout << root->item << " ";
    inorderTraversal(root->right);
}

void preorderTraversal(node* root) {
    if (!root) return;
    cout << root->item << " ";
    preorderTraversal(root->left);
    preorderTraversal(root->right);
}

void postorderTraversal(node* root) {
    if (!root) return;
    postorderTraversal(root->left);
    postorderTraversal(root->right);
    cout << root->item << " ";
}

node* create(int value) {
    node* newNode = new node; // C++ way (no malloc)
    newNode->item = value;
    newNode->left = nullptr;
    newNode->right = nullptr;
    return newNode;
}

node* insertNode(node* root, int value) {
    if (!root) return create(value);
    if (value < root->item) root->left = insertNode(root->left, value);
    else root->right = insertNode(root->right, value);
    return root;
}

int main() {
    node* root = nullptr;
    int n, value;

    cout << "By Rohan Kumar Pandey" << endl;
    cout << "Enter number of nodes: ";
    cin >> n;
```

```

for (int i = 0; i < n; i++) {
    cout << "Enter value for node " << i + 1 << ": ";
    cin >> value;
    root = insertNode(root, value);
}

cout << "\nInorder: ";
inorderTraversal(root);

cout << "\nPreorder: ";
preorderTraversal(root);

cout << "\nPostorder: ";
postorderTraversal(root);

cout << "\n";
}

```

By Rohan Kumar Pandey

Enter number of nodes: 4

Enter value for node 1: 10

Enter value for node 2: 32

Enter value for node 3: 20

Enter value for node 4: 40

Inorder: 10 20 32 40

Preorder: 10 32 20 40

Postorder: 20 40 32 10

=== Code Execution Successful ===

Binary Search Tree using Iteration

```

#include <iostream>
#include <stack>
using namespace std;

```

```

// Binary Tree Node
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

// Insert a node into BST
Node* insert(Node* root, int data) {
    if (!root) return new Node(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root;
}

// Iterative Inorder Traversal
void iterativeInorder(Node* root) {
    if (!root) return;

    stack<Node*> s;
    Node* current = root;

    while (current || !s.empty()) {
        while (current) {
            s.push(current);
            current = current->left;
        }
        current = s.top();
        s.pop();
        cout << current->data << " ";
        current = current->right;
    }
}

// Iterative Preorder Traversal
void iterativePreorder(Node* root) {
    if (!root) return;

    stack<Node*> s;
    s.push(root);

    while (!s.empty()) {
        Node* current = s.top();
        s.pop();
    }
}

```

```

        cout << current->data << " ";

        if (current->right) s.push(current->right);
        if (current->left) s.push(current->left);
    }
}

// Iterative Postorder Traversal
void iterativePostorder(Node* root) {
    if (!root) return;

    stack<Node*> s1, s2;
    s1.push(root);

    while (!s1.empty()) {
        Node* current = s1.top();
        s1.pop();
        s2.push(current);

        if (current->left) s1.push(current->left);
        if (current->right) s1.push(current->right);
    }

    while (!s2.empty()) {
        cout << s2.top()->data << " ";
        s2.pop();
    }
}

int main() {
    Node* root = nullptr;
    int n, data;

    cout<<"By Rohan Kumar Pandey" <<endl;
    cout << "Enter the number of nodes: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        cout << "Enter value for node " << i + 1 << ": ";
        cin >> data;
        root = insert(root, data);
    }

    cout << "Iterative Inorder Traversal: ";
    iterativeInorder(root);
    cout << "\n";

    cout << "Iterative Preorder Traversal: ";
    iterativePreorder(root);
    cout << "\n";

    cout << "Iterative Postorder Traversal: ";
    iterativePostorder(root);
    cout << "\n";
}

```

```
        return 0;
    }
```

By Rohan Kumar Pandey

Enter the number of nodes: 4

Enter value for node 1: 23

Enter value for node 2: 1

Enter value for node 3: 56

Enter value for node 4: 76

Iterative Inorder Traversal: 1 23 56 76

Iterative Preorder Traversal: 23 1 56 76

Iterative Postorder Traversal: 1 76 56 23

=== Code Execution Successful ===

Binary Search Tree using Recursion

```
#include <iostream>
using namespace std;
```

```
// Binary tree node
```

```
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};
```

```
// Insert a node in BST
```

```
Node* insert(Node* root, int data) {
```

```

    if (root == nullptr) {
        return new Node(data);
    }

    if (data < root->data) {
        root->left = insert(root->left, data);
    }
    else if (data > root->data) {
        root->right = insert(root->right, data);
    }

    return root;
}

// Recursive In-order Traversal
void inorderTraversal(Node* root) {
    if (root != nullptr) {
        inorderTraversal(root->left);
        cout << root->data << " ";
        inorderTraversal(root->right);
    }
}

// Recursive Pre-order Traversal
void preorderTraversal(Node* root) {
    if (root != nullptr) {
        cout << root->data << " ";
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}

// Recursive Post-order Traversal
void postorderTraversal(Node* root) {
    if (root != nullptr) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        cout << root->data << " ";
    }
}

int main() {

```

```
Node* root = nullptr;
int n, data;

cout<<"By Rohan Kumar Pandey"<<endl;
cout << "Enter the number of nodes: ";
cin >> n;

for (int i = 0; i < n; i++) {
    cout << "Enter value for node " << i + 1 << ": ";
    cin >> data;
    root = insert(root, data);
}

cout << "Recursive In-order Traversal: ";
inorderTraversal(root);
cout << "\n";

cout << "Recursive Pre-order Traversal: ";
preorderTraversal(root);
cout << "\n";

cout << "Recursive Post-order Traversal: ";
postorderTraversal(root);
cout << "\n";

return 0;
}
```

By Rohan Kumar Pandey

Enter the number of nodes: 4

Enter value for node 1: 25

Enter value for node 2: 73

Enter value for node 3: 64

Enter value for node 4: 70

Recursive In-order Traversal: 25 64 70 73

Recursive Pre-order Traversal: 25 73 64 70

Recursive Post-order Traversal: 70 64 73 25

=== Code Execution Successful ===